Pipeline Traces

ELL782 Assignment 2 · Dharun Thota (Entry No. 2026EET2401)

How to read these traces

- The traces use the two test programs of validation.pdf, where their full ROM listings are given. Cases 8.1–8.3 come from the Group 1 program; case 8.4 and the wrong-path HALT come from the Group 2 program. Both programs first load the registers with LDI, because registers and RAM are not reset.
- Cycle 0 is the first clock cycle after RESET. Cycle numbers and addresses therefore match the listings in validation.pdf.
- The values were read from the debug output pins of `main` (PC, the four valid bits and stage PCs, STALL, FLUSH, SEL_A, SEL_B and the register-write and store ports) while running the program.

Layout

- Each column is one clock cycle; each of the first five rows is a pipeline stage. A cell shows the instruction held in that stage: its address and assembly. IF shows the instruction being fetched; ID, EX, MEM and WB show the contents of IF/ID, ID/EX, EX/MEM and MEM/WB. *bubble* means V = 0.
- **V**: the valid bits of IF/ID, ID/EX, EX/MEM and MEM/WB. **STALL**, **FLUSH**: the HAZ outputs.
- **fwd A/B**: where the instruction in EX takes its operands. R = register value from ID/EX, M = EX/MEM result, W = MEM/WB value, – = operand not read.
- **Writes**: architectural writes at the rising edge that ends the cycle: a register write from WB or a memory or I/O write (M[address]) from MEM.

8.1 ALU forwarding, no stall: `ADD R1, R2; SUB R3, R1`

Architectural state before the first instruction of the case: R1 = 0x01, R2 = 0x02, R3 = 0x03. SUB reads R1 in EX one cycle after ADD computed it, so R1 is forwarded from EX/MEM (fwd A/B = R/M). No stall. Result R1 = 0x03, R3 = 0x03 − 0x03 = 0x00.

Trace 8.1 (Group 1 program)

Cycle	10	11	12	13	14	15
IF	0A ADD R1,R2	0B SUB R3,R1	0C NOP	0D NOP	0E NOP	0F LD R1,[R6]
ID	09 NOP	0A ADD R1,R2	0B SUB R3,R1	0C NOP	0D NOP	0E NOP
EX	08 NOP	09 NOP	0A ADD R1,R2	0B SUB R3,R1	0C NOP	0D NOP
MEM	07 NOP	08 NOP	09 NOP	0A ADD R1,R2	0B SUB R3,R1	0C NOP
WB	06 ST R5,[R6]	07 NOP	08 NOP	09 NOP	0A ADD R1,R2	0B SUB R3,R1
V	1111	1111	1111	1111	1111	1111
STALL	0	0	0	0	0	0
FLUSH	0	0	0	0	0	0
fwd A/B	-/-	-/-	R/R	R/M	-/-	-/-
Writes	–	–	–	–	R1←0x03	R3←0x00

8.2 Load-use, exactly one stall: `LD R1, [R6]; ADD R3, R1`

Architectural state before the first instruction of the case: R1 = 0x03, R3 = 0x00, R6 = 0x05, RAM[0x05] = 0x21. In the cycle where LD is in EX and ADD is in ID, HAZ raises STALL: PC and IF/ID hold, ID/EX receives a bubble, LD moves on to MEM. One cycle later the load value is in MEM/WB and ADD takes it from there (fwd A/B = R/W). Exactly one bubble; R1 = 0x21, R3 = 0x00 + 0x21 = 0x21.

Trace 8.2 (Group 1 program)

Cycle	**15**	**16**	**17**	**18**	**19**	**20**	**21**
IF	0F `LD R1,[R6]`	10 `ADD R3,R1`	11 (held)	11 `NOP`	12 `NOP`	13 `NOP`	14 `LDI R1,0x01`
ID	0E `NOP`	0F `LD R1,[R6]`	10 `ADD R3,R1`	10 `ADD R3,R1`	11 `NOP`	12 `NOP`	13 `NOP`
EX	0D `NOP`	0E `NOP`	0F `LD R1,[R6]`	bubble	10 `ADD R3,R1`	11 `NOP`	12 `NOP`
MEM	0C `NOP`	0D `NOP`	0E `NOP`	0F `LD R1,[R6]`	bubble	10 `ADD R3,R1`	11 `NOP`
WB	0B `SUB R3,R1`	0C `NOP`	0D `NOP`	0E `NOP`	0F `LD R1,[R6]`	bubble	10 `ADD R3,R1`
V	`1111`	`1111`	`1111`	`1011`	`1101`	`1110`	`1111`
STALL	0	0	**1**	0	0	0	0
FLUSH	0	0	0	0	0	0	0
fwd A/B	`-/-`	`-/-`	`-/R`	`-`	`R/W`	`-/-`	`-/-`
Writes	R3←0x00	–	–	–	R1←0x21	–	R3←0x21

8.3 Youngest producer to a store: `ADD R1, R2; SUB R1, R3; ST R1, [R4]`

Architectural state before the first instruction of the case: R1 = 0x01, R2 = 0x02, R3 = 0x01, R4 = 0x10. When ST is in EX, both EX/MEM (SUB, R1 = 0x02) and MEM/WB (ADD, R1 = 0x03) hold a value for R1; the EX/MEM one is younger and wins (fwd A = M), so the store data SD = 0x02, not the stale register value 0x01 or the older 0x03. The store commits once, in MEM: RAM[0x10] ← 0x02.

Trace 8.3 (Group 1 program)

Cycle	**26**	**27**	**28**	**29**	**30**	**31**	**32**
IF	19 `ADD R1,R2`	1A `SUB R1,R3`	1B `ST R1,[R4]`	1C `NOP`	1D `NOP`	1E `NOP`	1F `HALT`
ID	18 `NOP`	19 `ADD R1,R2`	1A `SUB R1,R3`	1B `ST R1,[R4]`	1C `NOP`	1D `NOP`	1E `NOP`
EX	17 `NOP`	18 `NOP`	19 `ADD R1,R2`	1A `SUB R1,R3`	1B `ST R1,[R4]`	1C `NOP`	1D `NOP`
MEM	16 `NOP`	17 `NOP`	18 `NOP`	19 `ADD R1,R2`	1A `SUB R1,R3`	1B `ST R1,[R4]`	1C `NOP`
WB	15 `LDI R3,0x01`	16 `NOP`	17 `NOP`	18 `NOP`	19 `ADD R1,R2`	1A `SUB R1,R3`	1B `ST R1,[R4]`
V	`1111`	`1111`	`1111`	`1111`	`1111`	`1111`	`1111`
STALL	0	0	0	0	0	0	0
FLUSH	0	0	0	0	0	0	0
fwd A/B	`-/-`	`-/-`	`R/R`	`M/R`	`M/R`	`-/-`	`-/-`
Writes	R3←0x01	–	–	–	R1←0x03	R1←0x02 M[0x10]←0x02	–

8.4 Taken branch kills a wrong-path actuator store: `SUB R1, R1; BEQZ R1, t1; ST R2, [R5]; ADD R3, R4`

Architectural state before the first instruction of the case: R2 = 0x01, R3 = 0x05, R4 = 0x06, R5 = 0xF4 (R5 = 0xF4 is the ACT address, Rvalve). BEQZ reads R1 = 0 forwarded from EX/MEM and is taken in EX. In that cycle FLUSH = 1: the ST in ID and the ADD being fetched become bubbles (two wrong-path instructions, two-cycle penalty) and the PC loads t1 = 0x0B. The ST never reaches MEM with V = 1, so ST_WE stays 0, ACT stays 0x00 and every actuator pin stays 0; R3 stays 0x05.

Trace 8.4 (Group 2 program)

Cycle	**7**	**8**	**9**	**10**	**11**	**12**
IF	07 `SUB R1,R1`	08 `BEQZ R1,0x0B`	09 `ST R2,[R5]`	0A `ADD R3,R4`	0B `SUB R1,R1`	0C `BEQZ R1,0x0F`
ID	06 `NOP`	07 `SUB R1,R1`	08 `BEQZ R1,0x0B`	09 `ST R2,[R5]`	bubble	0B `SUB R1,R1`
EX	05 `NOP`	06 `NOP`	07 `SUB R1,R1`	08 `BEQZ R1,0x0B`	bubble	bubble
MEM	04 `NOP`	05 `NOP`	06 `NOP`	07 `SUB R1,R1`	08 `BEQZ R1,0x0B`	bubble
WB	03 `LDI R5,0xF4`	04 `NOP`	05 `NOP`	06 `NOP`	07 `SUB R1,R1`	08 `BEQZ R1,0x0B`
V	`1111`	`1111`	`1111`	`1111`	`0011`	`1001`
STALL	0	0	0	0	0	0
FLUSH	0	0	0	**1**	0	0
fwd A/B	`-/-`	`-/-`	`R/R`	`M/-`	`-`	`-`
Writes	R5←0xF4	–	–	–	R1←0x00	–

Wrong-path HALT: `SUB R1, R1; BEQZ R1, t2; HALT; HALT; t2: LDI R7, 0x55; NOP; NOP; HALT`

The first HALT is in ID and the second is being fetched when BEQZ is taken in EX; both are flushed, so HALTED stays 0 and the machine continues at t2. Only the final HALT, valid in EX, sets HALTED: from that edge the PC holds and IF/ID stays invalid, while the two older NOPs in MEM and WB finish normally (LDI R7 has already written R7 = 0x55). The HALT itself then drains through MEM and WB with no effect.

Wrong-path HALT (Group 2 program)

Cycle	11	12	13	14	15	16	17	18	19	20	21	22
IF	0B SUB R1,R1	0C BEQZ R1,0x0F	0D HALT	0E HALT	0F LDI R7,0x55	10 NOP	11 NOP	12 HALT	13 NOP	14 NOP	14 (halted)	14 (halted)
ID	bubble	0B SUB R1,R1	0C BEQZ R1,0x0F	0D HALT	bubble	0F LDI R7,0x55	10 NOP	11 NOP	12 HALT	13 NOP	bubble	bubble
EX	bubble	bubble	0B SUB R1,R1	0C BEQZ R1,0x0F	bubble	bubble	0F LDI R7,0x55	10 NOP	11 NOP	12 HALT	bubble	bubble
MEM	08 BEQZ R1,0x0B	bubble	bubble	0B SUB R1,R1	0C BEQZ R1,0x0F	bubble	bubble	0F LDI R7,0x55	10 NOP	11 NOP	12 HALT	bubble
WB	07 SUB R1,R1	08 BEQZ R1,0x0B	bubble	bubble	0B SUB R1,R1	0C BEQZ R1,0x0F	bubble	bubble	0F LDI R7,0x55	10 NOP	11 NOP	12 HALT
V	0011	1001	1100	1110	0011	1001	1100	1110	1111	1111	0011	0001
STALL	0	0	0	0	0	0	0	0	0	0	0	0
FLUSH	0	0	0	**1**	0	0	0	0	0	**1**	0	0
fwd A/B	-	-	R/R	M/-	-	-	-/-	-/-	-/-	-/-	-	-
Writes	R1←0x00	–	–	–	R1←0x00	–	–	–	R7←0x55	–	–	–

Source-use detection

HAZ and FWD compare a register field only when the instruction really reads it. `USE_A` and `USE_B` come from the control unit (decoded from the opcode and I, isa.pdf section 6), never from the raw bits, because a field that is not an operand holds an immediate, a target, a reserved field or a destination. Using the raw bits would create false stalls.

Instruction class	Reads Rd [10:8] (USE_A)	Reads Rs [2:0] (USE_B)	Example that must *not* stall after `LD R1, [R2]`
NOP, HALT, JMP, reserved	no	no	`JMP 0x01`: target bits [2:0] = 001 look like Rs = R1
LDI	no	no	`LDI R1, 7`: R1 is only the destination
MOV, LD	no	yes	`MOV R1, R4`: R1 is only the destination
ADDI	yes	no	`ADDI R5, 0x01`: immediate bits [2:0] look like Rs = R1
BEQZ, BNEZ	yes	no	`BEQZ R5, 0x09`: target bits [2:0] look like Rs = R1
ADD, SUB, AND, SLTU, ST	yes	yes	both fields are real operands; ST reads data (Rd) and address (Rs)

Running these pairs on the processor confirms the rule: `LD; LDI` on the same register, `LD; MOV` into the loaded register, `LD; ADDI` whose immediate low bits equal the load target, and `LD; JMP` whose target low bits equal it all run with STALL = 0, while `LD R2, [R2]; LD R3, [R2]` (load-use on the address register), `LD; ST` (on the store data) and `LD; BNEZ` (on the branch test) each stall exactly once.

A store commits at most once

Only PC and IF/ID can hold; ID/EX, EX/MEM and MEM/WB load at every edge, and memory never stalls. A store therefore spends exactly one cycle in MEM, and the RAM and peripheral write strobes (ST_WE = EXMEM_V $\wedge$ EXMEM_isST, then gated by the address decode) are active only in that cycle, so the write lands at exactly one edge. While an instruction is held in ID by a stall, ID/EX receives a bubble, so the held instruction is not duplicated downstream. A flushed or bubbled store has V = 0 and never writes (trace 8.4).